

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Tamil Elakiya S	Reg. No.:	192524443
Date:	17/08/2026	Duration:	60 minutes

Code of Conduct

I Tamil Elakiya S (192524443) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tamil Elakiya S

Rubrics for Calculation

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Digital Farmer Marketplace for Agricultural Product Trading

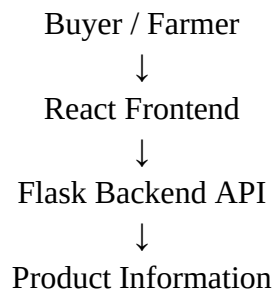
1. Problem Overview

1.1 Assigned Problem Statement

The assigned problem is to develop a **Digital Farmer Marketplace for Agricultural Product Trading** that provides a direct connection between farmers and buyers. Traditional agricultural product trading often involves intermediaries, limited market access, lack of price transparency, and difficulty in reaching customers.

The proposed system provides a software-based marketplace where agricultural products can be displayed digitally and accessed by buyers. The system is implemented as a web application with a **React-based frontend** and **Flask-based backend**. Docker is used to containerize the frontend and backend so that the application can be executed consistently across different environments.

The application follows a simple client-server structure:



The project is maintained using **Git and GitHub**, which allows the development work to be tracked through commits and branches.

1.2 Project Objectives

The major objectives of the project are:

- To provide a digital platform for agricultural product trading.
- To connect farmers directly with potential buyers.
- To display available agricultural products in an organized manner.
- To provide a simple and user-friendly interface.
- To develop a maintainable software structure.
- To use Git for version control and collaboration.
- To use Docker for application portability and deployment.

1.3 Key Functionalities

The implemented system includes:

- Digital Farmer Marketplace homepage.
- Agricultural product display.
- Farmer and buyer-oriented marketplace concept.
- Backend API for product information.
- React-based frontend interface.
- Flask backend service.
- Docker-based deployment.
- Git-based source-code management.

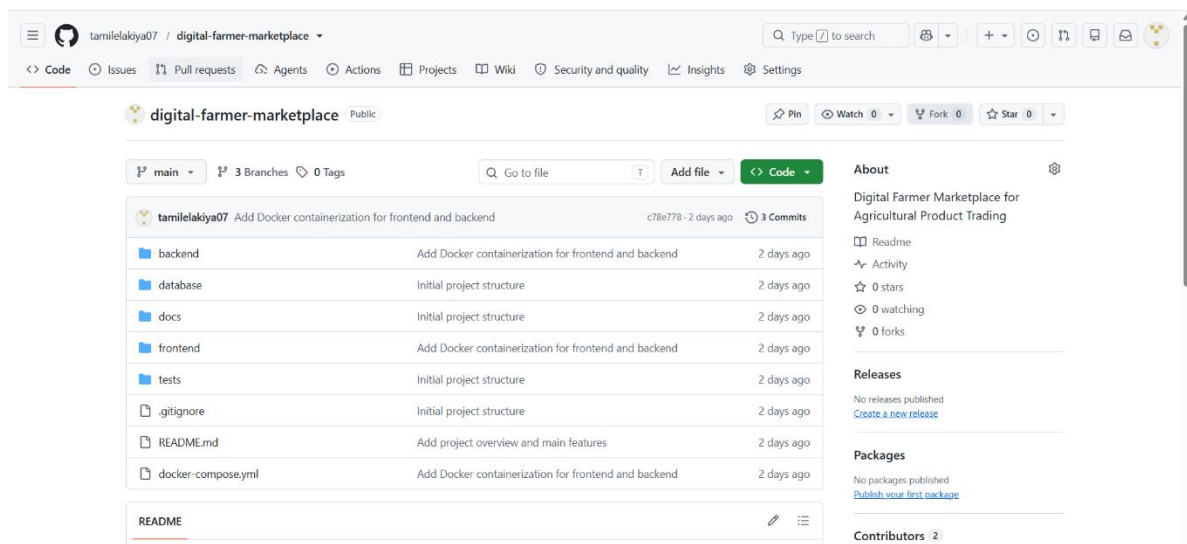


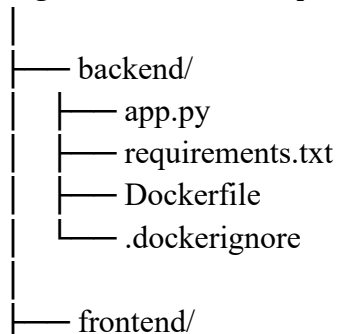
Figure 1: GitHub repository of the Digital Farmer Marketplace project

2. Repository Organization

2.1 Repository Structure

The project repository is organized into separate frontend and backend components.

Digital-Farmer-Marketplace/



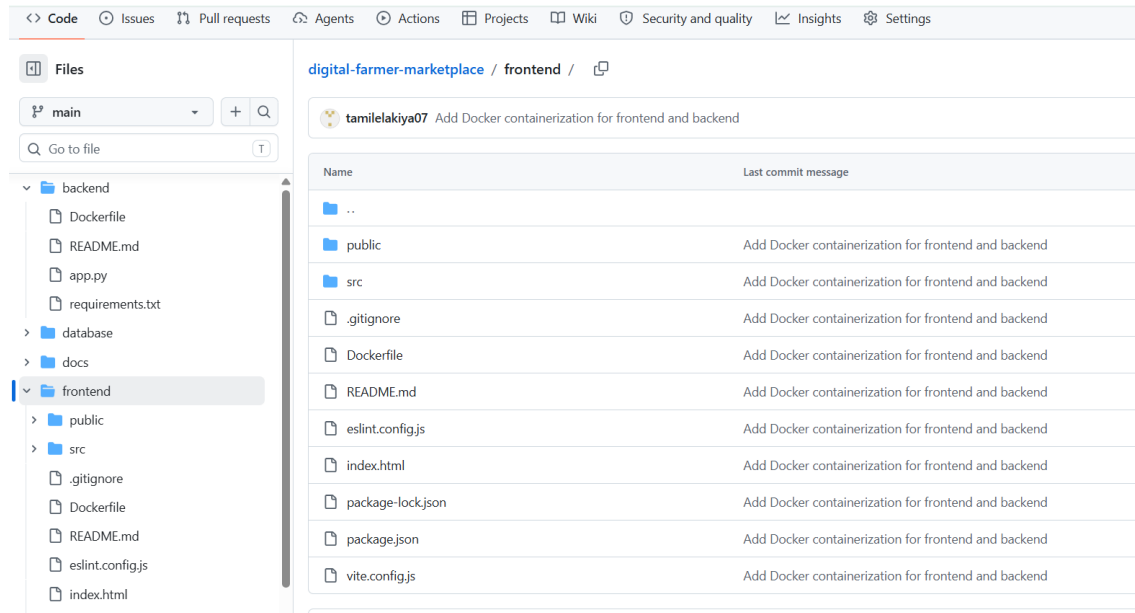
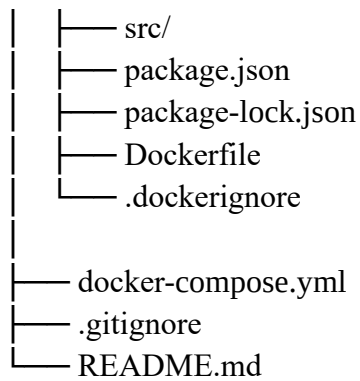


Figure 2: Project repository structure showing frontend, backend and Docker configuration

2.2 Folder Organization

Backend:

Contains the Flask application, Python dependencies and backend Docker configuration.

Frontend:

Contains the React/Vite application, source code, JavaScript/JSX files and frontend Docker configuration.

Docker Compose:

Defines and manages the frontend and backend containers.

.gitignore:

Prevents unnecessary files such as virtual environments and generated files from being committed to Git.

2.3 File Naming Conventions

The project uses meaningful names such as:

- app.py – backend application.
- requirements.txt – Python dependencies.
- package.json – frontend dependencies and scripts.
- Dockerfile – container build instructions.
- docker-compose.yml – multi-container configuration.

Meaningful naming makes the repository easier to understand and maintain.

2.4 Maintainability and Collaboration

The separation of frontend and backend makes the system modular. Developers can work independently on different components without unnecessarily modifying other parts of the project.

The repository structure also makes it easier to:

- Locate files.
- Identify application components.
- Add new features.
- Fix bugs.
- Maintain Docker configurations.
- Collaborate through Git branches.

3. Code Quality Review

3.1 Readability

The source code is organized into understandable frontend and backend components. Meaningful file names and a simple project structure make the code relatively easy to understand.

The frontend follows the React/Vite structure, while the backend uses Flask for API development.

3.2 Modularity

The project separates:

Frontend → User Interface

Backend → API and Application Logic

Docker → Deployment Configuration

Git → Version Management

This separation improves modularity because changes to one component can be made without completely redesigning the other.

```
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

@app.route("/")
def home():
    return jsonify({
        "message": "Digital Farmer Marketplace API is running"
    })

@app.route("/api/products")
def products():
    return jsonify([
        {
            "id": 1,
            "name": "Tomato",
            "price": 40,
            "unit": "kg"
        },
        {
```

Figure 3: Backend Flask implementation and API route

```
File Edit View

import { useEffect, useState } from "react";
import "./App.css";

function App() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("http://backend:5000/api/products")
      .then((response) => response.json())
      .then((data) => {
        setProducts(data);
        setLoading(false);
      })
      .catch((error) => {
        console.error("Error fetching products:", error);
        setLoading(false);
      });
  }, []);

  return (
    <div className="app">
      <header className="header">
        <div>
```

Figure 4: React frontend implementation of the Digital Farmer Marketplace

3.3 Coding Standards

The project uses:

- Python for backend development.
- JavaScript/JSX for frontend development.
- YAML for Docker Compose configuration.
- Standard Git commands for version control.

The use of established technologies and their standard project structures improves consistency.

3.4 Documentation

Basic comments and meaningful names help explain the purpose of files and components. However, additional documentation can be added to complex functions and API endpoints.

3.5 Strengths

The major strengths identified during the review are:

- Clear frontend-backend separation.
- Simple repository organization.
- Containerized application.
- Meaningful file names.
- Use of version control.
- Easy local deployment using Docker Compose.

3.6 Areas for Improvement

Possible improvements include:

- Adding more comments to complex logic.
- Separating business logic into dedicated backend modules as the project grows.
- Adding automated unit tests.
- Introducing environment variables for configuration.
- Improving API error handling.
- Adding stronger input validation.

4. Version Control Evaluation

4.1 Git Repository

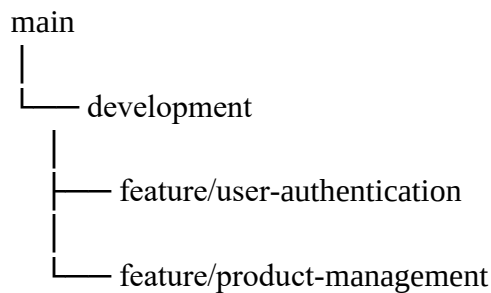
Git is used to track changes made throughout the project. The repository is hosted on GitHub.

GitHub Repository:

<https://github.com/tamilelakiya07/digital-farmer-marketplace>

4.2 Branching Strategy

The project uses multiple branches:



The **main** branch represents the stable version of the project.

The **development** branch is used for integrating ongoing development work.

Feature branches are used for implementing specific functionalities.

```
PS E:\Software engineering\Digital-Farmer-Marketplace> git branch -a
  development
  feature/product-management
  feature/user-authentication
* main
  remotes/origin/development
  remotes/origin/feature/user-authentication
  remotes/origin/main
PS E:\Software engineering\Digital-Farmer-Marketplace>
```

A screenshot of a terminal window showing the output of the 'git branch -a' command. The output lists the local branches: 'development', 'feature/product-management', 'feature/user-authentication', and 'main' (which is the current branch, indicated by an asterisk). It also lists the remote branches: 'remotes/origin/development', 'remotes/origin/feature/user-authentication', and 'remotes/origin/main'.

Figure 5: Git branching structure used in the project

4.3 Commit History

The project contains meaningful commits such as:

- Initial project structure
- Add project overview and main features
- Add Docker containerization for frontend and backend

- These messages clearly communicate what was changed.

```
PS E:\Software engineering\Digital-Farmer-Marketplace> git log --oneline --graph --all -10
* c78e778 (HEAD -> main, origin/main, origin/development, development) Add Docker containeriz
ation for frontend and backend
* 7b09f0f (origin/feature/user-authentication, feature/user-authentication, feature/product-m
anagement) Add project overview and main features
* 4e0bead Initial project structure
PS E:\Software engineering\Digital-Farmer-Marketplace> |
```

Figure 6: Git commit history showing incremental development

4.4 Version Control Benefits

Git provides:

- Change tracking.
- Backup of project history.
- Branch-based development.
- Collaboration between developers.
- Ability to restore previous versions.
- Easier debugging.
- Controlled integration of new features.

4.5 Repository Maintenance

Regular commits and meaningful commit messages make the project history easier to understand. Branch separation also reduces the risk of directly affecting the stable version of the application.

5. Code Testing and Validation

5.1 Testing Approach

The application was tested by running the frontend and backend separately and then testing them together using Docker Compose.

The major testing activities included:

1. Backend execution testing.
2. Frontend execution testing.
3. Docker image build testing.
4. Docker container testing.
5. API testing.
6. Frontend accessibility testing.

5.2 Test Cases

Test ID	Test Case	Expected Result	Result
TC01	Start Flask backend	Backend starts successfully	Pass
TC02	Start React frontend	Frontend loads successfully	Pass
TC03	Build Docker images	Images build without errors	Pass
TC04	Start Docker Compose	Both containers start	Pass
TC05	Open localhost:5173	Marketplace webpage appears	Pass
TC06	Access product API	Product data is returned	Pass
TC07	Check running containers	Frontend and backend containers are running	Pass

5.3 Docker Testing

The following command was used:

docker compose up

The successful execution showed:

farmer-marketplace-backend

farmer-marketplace-frontend

The frontend was available at:

<http://localhost:5173>

The backend API was accessed through:

<http://localhost:5001/api/products>



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
a1b9c928def2	digital-farmer-marketplace-backend	"python app.py"	2 days ago	Up 45 seconds	0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
5820d64aba4b	digital-farmer-marketplace-frontend	"docker-entrypoint.s..."	2 days ago	Up 45 seconds	5173/tcp
d0e8cfb20d10	agriconnect-backend	"python run.py"	5 days ago	Up 40 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
d7c8b96ef3a5	agriconnect-frontend	"docker-entrypoint.s..."	5 days ago	Up 45 seconds	0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp
dbccae267846	mysql:8.4	"docker-entrypoint.s..."	5 days ago	Up 45 seconds (healthy)	3306/tcp

Figure 7: Running frontend and backend Docker containers

5.4 Testing Evidence

The report should include screenshots of:

- Docker Compose execution.
- docker ps.
- Marketplace homepage.
- Product API response.
- Git commit history.

6. Repository Documentation

6.1 README

The repository should contain a README.md file explaining the project.

The README should include:

1. Project Title
2. Project Description
3. Objectives
4. Features
5. Technologies Used
6. Project Structure
7. Installation Instructions
8. Running the Application
9. Docker Instructions
10. Git Workflow
11. Future Enhancements

6.2 Installation Instructions

The documentation should explain the basic requirements:

- Git

- Docker Desktop
- Node.js
- Python

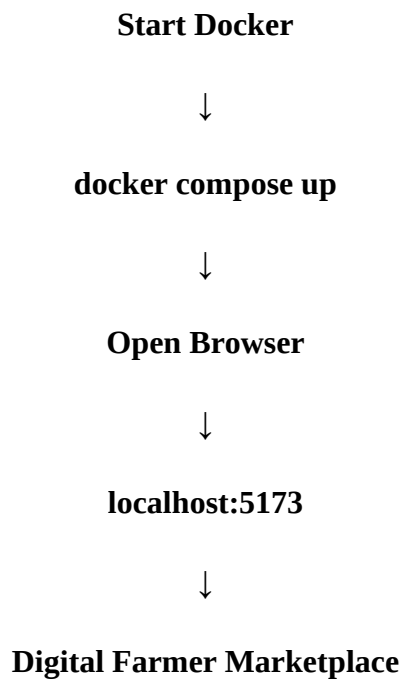
For Docker-based execution, the user can run:

- **docker compose build**
- **docker compose up**

6.3 Usage Guide

The documentation should explain how users access the application.

Example:



6.4 Dependencies

Backend dependencies are maintained in:

requirements.txt

Frontend dependencies are maintained in:

package.json

This allows dependencies to be installed consistently.

7. Code Review Findings and Recommendations

7.1 Overall Findings

The review shows that the Digital Farmer Marketplace has a **clear and functional basic software structure**. The separation of frontend and backend improves maintainability, while Git provides proper version control and Docker provides a consistent execution environment.

The project successfully demonstrates important software engineering practices such as:

- Modular design.
- Version control.
- Branch-based development.
- Containerization.
- API-based communication.
- Basic application testing.

7.2 Code Quality Recommendations

The following improvements are recommended:

- Introduce additional modular backend files.
- Add automated unit and integration tests.
- Improve error handling.
- Add stronger input validation.
- Use environment variables for configuration.
- Follow consistent formatting and linting rules.

7.3 Repository Management Recommendations

The repository can be improved by:

- Maintaining a clear branching policy.
- Using descriptive commit messages.
- Reviewing pull requests before merging.
- Regularly removing unused branches.
- Keeping sensitive information out of Git.

7.4 Documentation Recommendations

The README should be expanded with:

- Complete installation instructions.
- Screenshots.
- API usage.
- Docker instructions.
- Troubleshooting information.
- Contribution guidelines.

7.5 Testing Recommendations

Future testing should include:

- Unit testing.
- Integration testing.
- API testing.
- User acceptance testing.
- Performance testing.
- Security testing.

7.6 Future Enhancements

The Digital Farmer Marketplace can later include:

- Farmer registration and login.
- Buyer registration and authentication.
- Product listing and management.
- Online ordering.
- Secure payment integration.
- Order tracking.
- Farmer-buyer messaging.
- Product search and filtering.
- Location-based farmer discovery.
- Ratings and reviews.
- Database integration.
- Admin dashboard.